

Puntatori in C

Simone Remoli

Corso di Ingegneria Informatica

Indice

1	Prefazione	2
2	I puntatori	3
2.1	Il puntatore generico <code>void*</code>	4
2.2	Puntatori a funzione	7
2.3	Array di tripli puntatori che puntano ad array di doppi puntatori	10
2.4	Fusione e ordinamento di due vettori tramite doppi puntatori <code>void**</code>	13
2.5	Funzione che calcola la lunghezza precisa di una stringa tra- mite puntatore	19
2.6	Vettore di puntatori a carattere	21
2.7	Suddivisione di una stringa con delimitatori multipli	25
2.8	Esempio compatto/completo con <code>void *</code> , <code>restrict</code> e punta- tori a funzione	28
2.9	Albero di puntatori con quattro livelli di indirezione	33
2.10	IMPORTANTE: Ricorsione con puntatori a funzione e punta- tori generici <code>void*</code>	36
2.11	Esercizio MIT sui puntatori	40
2.12	Esempio MIT sull'aritmetica dei puntatori	40
2.13	Stanford University - Scambio degli estremi di un array tra- mite aritmetica dei puntatori	45
2.14	Dartmouth College - Array di strutture e aritmetica dei pun- tatori	47
3	Conclusione	52

1 Prefazione

Questo documento nasce dal desiderio di proporre una raccolta di esempi mirati all'apprendimento dei puntatori nel linguaggio C. Non ha l'obiettivo di sostituire un intero corso universitario, né di trattare in modo esaustivo tutti gli aspetti della programmazione in C, ma intende concentrarsi su uno degli argomenti più importanti, delicati e spesso temuti di questo linguaggio. La realizzazione di questo lavoro ha richiesto tempo, studio e revisione. Non si tratta di un documento nato dal nulla, ma di un percorso costruito progressivamente negli anni, tra impegni universitari, approfondimenti personali, esercizi, verifiche e consultazione di fonti autorevoli. Al suo interno sono presenti anche esempi ispirati a materiali didattici di università americane, scelti perché particolarmente significativi per comprendere in profondità il funzionamento dei puntatori.

L'obiettivo principale è fornire al lettore frammenti di codice mirati, ordinati e commentati, capaci di evidenziare concetti specifici. Ogni esempio è pensato non solo per mostrare la sintassi corretta, ma soprattutto per chiarire il ragionamento che si trova dietro l'uso degli indirizzi di memoria, della dereferenziazione e dell'aritmetica dei puntatori.

Lo scopo non è imparare meccanicamente il codice, ma sviluppare la capacità di leggerlo, interpretarlo, modificarlo e comprenderne il comportamento in memoria. Attraverso esempi progressivi, il lettore potrà familiarizzare con puntatori semplici, doppi e tripli puntatori, puntatori generici, puntatori a funzione, array, stringhe, strutture, memoria dinamica e forme più avanzate di indirizzazione.

Se affrontato con attenzione e compreso nei suoi passaggi fondamentali, questo documento può rappresentare una base solida per rendere i puntatori un argomento finalmente chiaro e padroneggiabile.

2 I puntatori

Un **puntatore** in C è una variabile il cui valore non rappresenta direttamente un dato, ma l'indirizzo di memoria in cui tale dato è memorizzato.

```
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;

    printf("Indirizzo di x: %p\n", &x);
    //Indirizzo di x: 0x7ff7ba5e5ec8
    printf("Indirizzo contenuto in p: %p\n", p);
    //Indirizzo contenuto in p: 0x7ff7ba5e5ec8
    printf("Valore puntato da p: %d\n", *p);
    //Valore puntato da p: 10

    return 0;
}
```

In questo semplice esempio, la variabile puntatore **p** contiene l'indirizzo di memoria della variabile **x**. Attraverso l'operazione di **dereferenziazione**, indicata con ***p**, è possibile accedere al **valore** memorizzato all'indirizzo puntato da **p**.

Di seguito un esempio di un doppio puntatore.

```
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;

    printf("Indirizzo di x: %p\n", &x);
    //Indirizzo di x: 0x7ff7ba5e5ec8
    printf("Indirizzo contenuto in p: %p\n", p);
    //Indirizzo contenuto in p: 0x7ff7ba5e5ec8
    printf("Valore puntato da p: %d\n", *p);
    //Valore puntato da p: 10

    int **ptr = &p;
    printf("Indirizzo del doppio puntatore = %p\n", &ptr);
    //Indirizzo del doppio puntatore = 0x7ff7b22d9eb8
    printf("Indirizzo a cui punta il doppio puntatore = %p\n", *ptr)
    ;
}
```

```

    //Indirizzo a cui punta il doppio puntatore = 0x7ff7ba5e5ec8
    printf("Valore finale a cui punta il doppio puntatore = %d\n",
    **ptr);
    //Valore finale a cui punta il doppio puntatore = 10

    return 0;
}

```

Nel codice viene dichiarata una variabile intera **x**, alla quale viene assegnato il valore 10. Successivamente viene dichiarato un puntatore a intero, **p**, che contiene l'indirizzo di memoria della variabile **x**. La stampa di **&x** mostra l'indirizzo effettivo della variabile **x**, mentre la stampa di **p** mostra il valore contenuto nel puntatore. Poiché **p** punta a **x**, i due indirizzi coincidono. Attraverso l'espressione ***p** si effettua la dereferenziazione del puntatore, cioè si accede al valore memorizzato nella variabile puntata da **p**. In questo caso, il valore ottenuto è 10. Successivamente viene introdotto un doppio puntatore, **ptr**, che contiene l'indirizzo del puntatore **p**. In altre parole, **ptr** non punta direttamente alla variabile **x**, ma punta alla variabile puntatore **p**. La stampa di **&ptr** mostra l'indirizzo di memoria del doppio puntatore stesso. La stampa di ***ptr**, invece, mostra il valore contenuto nella variabile puntata da **ptr**, cioè il contenuto di **p**. Poiché **p** contiene l'indirizzo di **x**, il valore stampato coincide ancora con l'indirizzo della variabile **x**. Infine, l'espressione ****ptr** applica una doppia dereferenziazione: prima si accede a **p**, poi si accede alla variabile puntata da **p**, cioè **x**. Per questo motivo, il valore finale stampato è ancora 10.

2.1 Il puntatore generico void*

Un **puntatore generico** in C, dichiarato come **void ***, è un puntatore capace di contenere l'indirizzo di memoria di un oggetto di qualunque tipo. A differenza di un puntatore tipizzato, come **int *** o **char ***, un puntatore **void *** non conosce il tipo del dato a cui punta. Per questo motivo, non può essere dereferenziato direttamente: prima di accedere al valore puntato, è necessario convertirlo esplicitamente nel tipo corretto. I puntatori generici sono molto utili quando si vogliono scrivere funzioni flessibili, capaci di lavorare con dati di tipo diverso.

```

#include <stdio.h>
#include <stdlib.h>

void load(void*, int);
void views(void*, void*);

```

```

int main(int argc, char **argv)
{
    int dim = 10;

    // Alloco dinamicamente
    //un array di 10 interi.
    int *ptr = malloc(sizeof(int) * dim);

    // Salvo l'indirizzo della
    //variabile dim dentro
    //un puntatore generico.
    void *ptr_dim = &dim;

    // Passo l'array alla funzione load.
    // ptr_dim viene riconvertito
    //a int* per ottenere il valore di dim.
    load(ptr, *(int*)ptr_dim-1);

    // Passo sia l'array sia la
    //dimensione alla funzione views.
    // Entrambi vengono passati
    //come void*, quindi in modo generico.
    views(ptr, ptr_dim);

    free(ptr);
    //libero la memoria

    return 0;
}

void load(void *arg, int i)
{
    printf(" ");

    // Riconverto il puntatore
    // generico a puntatore a intero.
    int *p = arg;

    // Scrivo il valore 1 nella
    // posizione i-esima dell'array.
    *(p + i) = 1;

    // Caso base della ricorsione.

```

```

    if (i == -1)
        return;
    else
        // Richiamo la funzione
        // sulla posizione precedente.
        load(arg, i - 1);
}

void views(void *arg, void *arg2)
{
    printf("\n");

    // Riconverto il primo puntatore generico nell'array di interi.
    int *scan = arg;

    // Riconverto il secondo puntatore generico a int*
    // e recupero il valore della dimensione.
    int dimensione = *(int*)arg2;

    // Scorro l'array e stampo ogni elemento.
    for (int i = 0; i < dimensione; i++)
    {
        printf(" %d \n", *(scan + i));
    }
}

```

In questo esempio viene mostrato l'utilizzo di un puntatore generico `void *` per passare a una funzione dati di tipo diverso. L'array dinamico viene riempito ricorsivamente attraverso una funzione che riceve un puntatore generico, lo riconverte nel tipo corretto e accede agli elementi tramite aritmetica dei puntatori. La seconda funzione utilizza lo stesso principio per leggere e stampare il contenuto dell'array, recuperando separatamente anche la sua dimensione. L'esempio evidenzia quindi come `void *` permetta di scrivere funzioni più generiche, a patto di effettuare correttamente le conversioni di tipo prima di accedere ai dati.

2.2 Puntatori a funzione

Un **puntatore a funzione** in C è un puntatore che contiene l'indirizzo di una funzione, invece dell'indirizzo di una variabile. Permette quindi di richiamare una funzione in modo indiretto, scegliendo a runtime quale funzione eseguire.

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>

/*
 * Prototipi delle funzioni che verranno richiamate
 * tramite puntatore a funzione.
 */
int add(int, int);
int sub(int, int);

/*
 * La funzione operazione riceve:
 * - due interi;
 * - un puntatore a funzione che prende due interi
 *   e restituisce un intero;
 * - una stringa che indica quale operazione eseguire.
 *
 * Restituisce un puntatore generico void*,
 * che contiene l'indirizzo del risultato calcolato.
 */
void* operazione(int, int, int (*)(int, int), char*);

int main(int argc, char **argv)
{
    /*
     * Dichiarazione di un puntatore a funzione.
     * ptrf puo puntare a una funzione che riceve due int
     * e restituisce un int.
     */
    int (*ptrf)(int, int);

    int somma, sottrazione;

    /*
     * Chiamo la funzione operazione chiedendo una somma.
     * Il risultato restituito e' un void*, quindi lo converto
     * a int* e poi lo dereferenzio per ottenere il valore intero.
     */
    somma = *(int*)operazione(3, 2, ptrf, "somma");

    /*
     * Chiamo la funzione operazione chiedendo una sottrazione.
     */
    sottrazione = *(int*)operazione(3, 2, ptrf, "sottrazione");
}

```

```

    printf("La somma = %d \n", somma); // out: La somma = 5
    printf("La sottrazione = %d \n", sottrazione); // out: La
    sottrazione = 1

    return 0;
}

/*
 * Funzione che somma due interi.
 */
int add(int a, int b)
{
    return a + b;
}

/*
 * Funzione che sottrae due interi.
 */
int sub(int a, int b)
{
    return a - b;
}

void* operazione(int a, int b, int (*p)(int, int), char *op)
{
    /*
     * Alloco dinamicamente lo spazio per il risultato.
     * In questo modo il valore rimane valido anche dopo
     * la fine della funzione.
     */
    int *risultato = malloc(sizeof(int));

    /*
     * Se la stringa op e' uguale a "somma",
     * assegno al puntatore a funzione p l'indirizzo
     * della funzione add.
     */
    if (strcmp(op, "somma") == 0)
    {
        p = add;

        /*

```



```

        * Richiamo la funzione tramite puntatore.
        * (*p)(a, b) equivale a p(a, b).
        */
    *risultato = (*p)(a, b);
}

/*
 * Se la stringa op e' uguale a "sottrazione",
 * assegno al puntatore a funzione p l'indirizzo
 * della funzione sub.
 */
if (strcmp(op, "sottrazione") == 0)
{
    p = sub;

    /*
     * Anche qui richiamo la funzione tramite puntatore.
     */
    *risultato = (*p)(a, b);
}

/*
 * Restituisco il risultato come puntatore generico.
 */
return risultato;
}

```

Nel codice viene mostrato un esempio di utilizzo dei puntatori a funzione in C. Le funzioni `add` e `sub` rappresentano due operazioni diverse, rispettivamente somma e sottrazione, ma condividono la stessa firma: entrambe ricevono due interi e restituiscono un intero. La funzione `operazione` riceve come parametro anche un puntatore a funzione, indicato con `int (*p)(int, int)`. In base alla stringa ricevuta, tale puntatore viene associato alla funzione `add` oppure alla funzione `sub`. In questo modo, la funzione da eseguire viene scelta durante l'esecuzione del programma. Il risultato dell'operazione viene allocato dinamicamente tramite `malloc` e restituito come puntatore generico `void *`. Nel `main`, tale puntatore viene riconvertito a `int *` e dereferenziato per ottenere il valore finale. L'esempio mostra quindi come i puntatori a funzione permettano di rendere il codice più flessibile, consentendo di selezionare dinamicamente il comportamento da eseguire senza modificare la struttura generale del programma.

2.3 Array di tripli puntatori che puntano ad array di doppi puntatori

```
void accesso(void*);

int main(int argc, char **argv)
{
    /*
     * general e' un triplo puntatore.
     * Lo usiamo per rappresentare una struttura composta da 2 basi.
     * Ogni base conterra 5 puntatori a intero.
     */
    int ***general = (int***)malloc(sizeof(int**) * 2);

    /*
     * Per ciascuna delle 2 basi, allochiamo spazio per 5 puntatori
     a int.
     */
    for (int i = 0; i < 2; i++)
    {
        general[i] = (int**)malloc(sizeof(int*) * 5);
    }

    /*
     * Dichiarazione di 10 variabili intere.
     * Le prime 5 verranno associate alla base 0,
     * le altre 5 alla base 1.
     */
    int a = 0; int f = 5;
    int b = 1; int g = 6;
    int c = 2; int h = 7;
    int d = 3; int i = 8;
    int e = 4; int l = 9;

    /*
     * Ogni elemento di general non contiene direttamente un valore
     intero,
     * ma l'indirizzo di una variabile intera.
     *
     * general[0] rappresenta la prima base.
     * general[1] rappresenta la seconda base.
     */
    general[0][0] = &a; general[1][0] = &f;
```

```

general[0][1] = &b; general[1][1] = &g;
general[0][2] = &c; general[1][2] = &h;
general[0][3] = &d; general[1][3] = &i;
general[0][4] = &e; general[1][4] = &l;

/*
 * Accesso diretto al valore puntato da general[0][2].
 * general[0][2] contiene l'indirizzo di c,
 * quindi *general[0][2] restituisce il valore di c.
 */
printf("Valore di c = %d \n", *general[0][2]);
//valore di c = 2.

/*
 * Scorriamo la prima base e passiamo ogni puntatore
 * alla funzione accesso.
 */
for (int i = 0; i < 5; i++)
    accesso(general[0][i]);

puts(" ");

/*
 * Scorriamo la seconda base e passiamo ogni puntatore
 * alla funzione accesso.
 */
for (int i = 0; i < 5; i++)
    accesso(general[1][i]);

/*
 * In un programma completo sarebbe opportuno liberare
 * la memoria allocata dinamicamente.
 */
for (int i = 0; i < 2; i++)
{
    free(general[i]);
}
free(general);

return 0;
}

void accesso(void *arg)

```

```

{
    /*
     * La funzione riceve un puntatore generico void*.
     * Poiche sappiamo che punta a un intero,
     * lo convertiamo a int* e poi lo dereferenziamo.
     */
    int value = *(int*)arg;

    /*
     * Stampa del valore ottenuto.
     */
    printf("Valore di value = %d \n", value);
}

```

La riga `int ***general = (int***)malloc(sizeof(int**) * 2);` alloca dinamicamente un array composto da due elementi, ciascuno dei quali è un puntatore di tipo `int **`. La variabile `general` è quindi un triplo puntatore: essa punta al primo elemento di una struttura che, a sua volta, conterrà due blocchi separati. In questo caso, `general[0]` e `general[1]` rappresentano i due blocchi principali della struttura. Tuttavia, questa prima allocazione crea soltanto il livello più esterno: per poter utilizzare effettivamente gli elementi interni, sarà necessario allocare successivamente anche la memoria per ciascun blocco. Alla luce di questa premessa, il codice costruisce una struttura dinamica basata su un triplo puntatore `int ***general`. Tale struttura può essere interpretata come un contenitore principale composto da due blocchi distinti, indicizzati come `general[0]` e `general[1]`. Ogni blocco contiene cinque puntatori a intero. Questo significa che gli elementi della struttura non memorizzano direttamente valori numerici, ma indirizzi di memoria di variabili intere dichiarate nel programma. In particolare, `general[0]` contiene i puntatori alle variabili `a`, `b`, `c`, `d`, `e`, mentre `general[1]` contiene i puntatori alle variabili `f`, `g`, `h`, `i`, `l`. L'espressione `general[0][2]` permette di accedere al terzo puntatore contenuto nel primo blocco. Poiché tale puntatore contiene l'indirizzo della variabile `c`, l'espressione `*general[0][2]` consente di dereferenziare quel puntatore e ottenere il valore effettivo di `c`. Successivamente, il programma attraversa separatamente i due blocchi tramite due cicli `for`. A ogni iterazione viene passato alla funzione `accesso` un puntatore a intero, ma il parametro della funzione è dichiarato come `void *`. Per questo motivo, all'interno della funzione il puntatore generico viene riconvertito a `int *` e poi dereferenziato per recuperare il valore intero. L'esempio mostra quindi come sia possibile utilizzare una struttura a più livelli di puntatori per organizzare gruppi di indirizzi di memoria e accedere indirettamente ai valori contenuti nelle variabili originali.

2.4 Fusione e ordinamento di due vettori tramite doppi puntatori void**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <math.h>

/*
 * Prototipi delle funzioni utilizzate nel programma.
 */
void load(void*, int);
void views(void*, int);
void sort(void*, int);
void swap(int*, int*);
void operation(void*, void*, void*, int, int);

int main(int argc, char **argv)
{
    /*
     * uso e' un array di puntatori generici.
     * Conterra' gli indirizzi dei due vettori di partenza
     * e del vettore finale.
     */
    void **uso = (void**)malloc(sizeof(void*) * 3);

    /*
     * double_array e' un array di puntatori a intero.
     * Lo usiamo per gestire:
     * - i due vettori di partenza;
     * - le loro dimensioni;
     * - il vettore finale.
     */
    int **double_array = malloc(sizeof(int*) * 6);

    /*
     * double_array[2] e double_array[3] contengono
     * rispettivamente la dimensione del primo e del secondo vettore
     */
    double_array[2] = malloc(sizeof(int) * 1);
    double_array[3] = malloc(sizeof(int) * 1);
```

```

printf("Inserisci la dimensione del primo vettore: ");
scanf("%d", &double_array[2][0]);

printf("Inserisci la dimensione del secondo vettore: ");
scanf("%d", &double_array[3][0]);

/*
 * Calcolo la dimensione complessiva del vettore finale.
 */
int dim = double_array[2][0] + double_array[3][0];

/*
 * Alloco dinamicamente i due vettori di partenza
 * e il vettore finale.
 */
double_array[0] = malloc(sizeof(int) * double_array[2][0]);
double_array[1] = malloc(sizeof(int) * double_array[3][0]);
double_array[5] = malloc(sizeof(int) * dim);

/*
 * Salvo gli indirizzi dei vettori dentro l'array uso.
 * Ogni elemento di uso e' un void*, quindi puo' contenere
 * l'indirizzo di dati di tipo diverso.
 */
uso[0] = &double_array[0][0];
uso[1] = &double_array[1][0];
uso[2] = &double_array[5][0];

/*
 * Carico i due vettori leggendo i valori da input.
 */
for (int i = 0; i < 2; i++)
    load(uso[i], double_array[i + 2][0]);

/*
 * Stampo i due vettori appena caricati.
 */
for (int i = 0; i < 2; i++)
    views(uso[i], double_array[i + 2][0]);

printf("Array ordinati: \n");

```

```

    /*
     * Ordino separatamente i due vettori.
     */
    for (int i = 0; i < 2; i++)
        sort(uso[i], double_array[i + 2][0]);

    /*
     * Stampo i due vettori dopo l'ordinamento.
     */
    for (int i = 0; i < 2; i++)
        views(uso[i], double_array[i + 2][0]);

    /*
     * Fondo i due vettori nel terzo vettore e poi lo ordino.
     */
    operation(uso[0], uso[1], uso[2], double_array[2][0],
double_array[3][0]);

    return 0;
}

void operation(void *ptrone, void *ptrtwo, void *ptrt, int dim1, int
dim2)
{
    /*
     * Riconverto i puntatori generici nei rispettivi tipi corretti.
     */
    int *ptr1 = ptrone;
    int *ptr2 = ptrtwo;
    int *ptr_tot = ptrt;

    /*
     * Copio il primo vettore nel vettore totale.
     */
    for (int i = 0; i < dim1; i++)
        *(ptr_tot + i) = *(ptr1 + i);

    /*
     * Copio il secondo vettore subito dopo il primo.
     */
    for (int i = dim1, j = 0; i < dim1 + dim2; i++, j++)
        *(ptr_tot + i) = *(ptr2 + j);
}

```

```

printf("Vettore totale: \n");
views(ptr_tot, dim1 + dim2);

/*
 * Ordino il vettore totale.
 */
sort(ptr_tot, dim1 + dim2);

printf("Vettore ordinato: \n");
views(ptr_tot, dim1 + dim2);
}

void swap(int *i, int *j)
{
    /*
     * Scambio il contenuto di due variabili intere
     * usando i loro indirizzi.
     */
    int temp = *i;
    *i = *j;
    *j = temp;
}

void sort(void *ptr, int dim)
{
    /*
     * Riconverto il puntatore generico a puntatore a intero.
     */
    int *array = ptr;

    /*
     * Ordinamento semplice tramite confronto tra coppie di elementi
     */
    for (int i = 0; i < dim; i++)
    {
        for (int j = i + 1; j < dim; j++)
        {
            if (array[i] > array[j])
            {
                swap(&array[i], &array[j]);
            }
        }
    }
}

```



```

    }
}

void views(void *ptr, int dim)
{
    /*
     * Riconverto il puntatore generico a puntatore a intero.
     */
    int *array = ptr;

    /*
     * Stampo tutti gli elementi del vettore.
     */
    puts(" ");
    for (int i = 0; i < dim; i++)
    {
        printf(" %d ", *(array + i));
    }
    printf("\n");
    puts(" ");
}

void load(void *ptr, int dim)
{
    /*
     * Riconverto il puntatore generico a puntatore a intero.
     */
    int *array = ptr;
    int val = 0;

    /*
     * Leggo da input i valori del vettore.
     */
    for (int i = 0; i < dim; i++)
    {
        scanf("%d", &val);
        *(array + i) = val;
    }

    puts(" ");
}

```

Il programma realizza la fusione di due vettori di interi attraverso l'utilizzo di memoria dinamica, puntatori doppi e puntatori generici void *. L'obiet-

tivo è leggere due vettori, memorizzarli dinamicamente, unirli in un terzo vettore e infine ordinare il risultato complessivo. La variabile `uso` è un array di puntatori generici utilizzato per conservare gli indirizzi dei due vettori di partenza e del vettore finale. In questo modo, le funzioni `load`, `views`, `sort` e `operation` possono ricevere i vettori tramite parametri di tipo `void *`, riconvertendoli poi internamente nel tipo corretto, cioè `int *`. La variabile `double_array`, invece, è un array di puntatori a intero. Alcune sue posizioni vengono usate per contenere i vettori veri e propri, mentre altre vengono usate per memorizzare le dimensioni dei due vettori. Dopo aver letto da input le due dimensioni, il programma alloca dinamicamente lo spazio necessario per il primo vettore, per il secondo vettore e per il vettore risultante. Le funzioni `load` e `views` si occupano rispettivamente del caricamento e della visualizzazione dei vettori. La funzione `sort` ordina un vettore tramite confronti successivi tra elementi, utilizzando la funzione `swap` per scambiare due valori quando necessario. La funzione `operation` riceve i due vettori iniziali e il vettore finale. Essa copia prima tutti gli elementi del primo vettore nel vettore risultante e poi aggiunge, in coda, tutti gli elementi del secondo vettore. Successivamente il vettore ottenuto viene ordinato, producendo così una sequenza unica crescente. L'esempio mostra quindi come sia possibile combinare memoria dinamica, aritmetica dei puntatori e puntatori generici per costruire una soluzione flessibile. Tuttavia, dal punto di vista algoritmico, questa soluzione non sfrutta pienamente il fatto che i due vettori iniziali siano già ordinati: una fusione più efficiente potrebbe confrontare progressivamente gli elementi dei due vettori, evitando di ordinare nuovamente l'intero vettore finale.

2.5 Funzione che calcola la lunghezza precisa di una stringa tramite puntatore

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <limits.h>
#include <stdint.h>

/*
 * Prototipo della funzione che calcola manualmente
 * la lunghezza di una stringa.
 */
int lunghezza_stringa(char*);
```

```

int main(int argc, char **argv)
{
    /*
     * Dichiaro un array di caratteri di dimensione 1024.
     * Questo array conterra' la stringa inserita dall'utente.
     */
    char mamma[1024];

    /*
     * fgets legge una riga da stdin e la salva dentro mamma.
     * Il secondo parametro indica la dimensione massima leggibile.
     */
    if (fgets(mamma, 1024, stdin))
    {
        /*
         * fgets salva anche il carattere '\n' se trova un invio.
         * Con strchr cerco la posizione del primo '\n'
         * e lo sostituisco con il terminatore di stringa '\0'.
         */
        mamma[strchr(mamma, "\n")] = '\0';
    }

    /*
     * Stampo la lunghezza della stringa calcolata dalla funzione.
     */
    printf("Dimensione di mamma = %d \n", lunghezza_stringa(mamma));

    return 0;
}

int lunghezza_stringa(char *s)
{
    /*
     * count conta il numero di caratteri attraversati.
     */
    int count = 0;

    /*
     * p punta all'inizio della stringa ricevuta.
     */
    char *p = s;

```

```

/*
 * Scorro la stringa finche' non incontro il terminatore '\0'.
 * In C una stringa termina sempre con il carattere nullo.
 */
while (*p != '\0')
{
    /*
     * Avanzo il puntatore al carattere successivo.
     */
    p++;

    /*
     * Incremento il contatore dei caratteri.
     */
    count = count + 1;
}

/*
 * Restituisco il numero totale di caratteri letti.
 */
return count;
}

```

Il programma mostra una possibile implementazione manuale del calcolo della lunghezza di una stringa in C, senza utilizzare direttamente la funzione `strlen` della libreria standard. Nel `main` viene dichiarato un array di caratteri `mamma` di dimensione 1024, utilizzato per memorizzare la stringa inserita dall'utente. La funzione `fgets` legge una riga da standard input e la salva all'interno dell'array, rispettando il limite massimo di caratteri specificato. Un aspetto importante riguarda la gestione del carattere di nuova riga. Quando l'utente preme Invio, `fgets` può memorizzare anche il carattere `\n` all'interno della stringa. Per evitare che tale carattere venga considerato parte effettiva del testo, viene utilizzata la funzione `strcspn`, che individua la posizione del primo carattere `\n`; tale posizione viene poi sostituita con il terminatore di stringa `\0`. La funzione `lunghezza_stringa` riceve come parametro un puntatore a carattere, cioè l'indirizzo del primo elemento della stringa. Al suo interno viene dichiarato un puntatore `p`, inizializzato all'inizio della stringa. Il puntatore viene poi fatto avanzare carattere per carattere finché non incontra il terminatore `\0`, che in C indica la fine della stringa. A ogni avanzamento del puntatore viene incrementato un contatore. Quando il ciclo termina, il contatore rappresenta il numero di caratteri effettivamente presenti nella stringa, escluso il terminatore finale. L'esempio evidenzia quindi come una stringa in C possa essere attraversata tramite aritmetica dei

puntatori e come il carattere nullo `\0` sia essenziale per determinarne la fine.

2.6 Vettore di puntatori a carattere

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXLINE 5000
#define MAXLEN 1000

/*
 * Prototipo della funzione che stampa le stringhe memorizzate.
 */
void writelines(char**, int);

int main(int argc, char **argv)
{
    /*
     * Numero di stringhe che vogliamo leggere da input.
     */
    int num_arrays = 5;

    /*
     * Dichiarazione di un doppio puntatore a char.
     * Questo verra' usato come array di puntatori a stringhe.
     */
    char **array_di_pointers_a_carattere;

    /*
     * Buffer temporaneo usato per leggere ogni riga da input.
     */
    char buffer[1024];

    /*
     * Alloco dinamicamente un array di 5 puntatori a char.
     * Ogni elemento puntera' poi a una stringa.
     */
    array_di_pointers_a_carattere =
        (char**)malloc(sizeof(char*) * num_arrays);

    /*
     * Per ogni posizione dell'array, alloco spazio per una stringa

```

```

    * e poi leggo una riga da input.
    */
for (int i = 0; i < num_arrays; i++)
{
    /*
     * Alloco 1024 caratteri per la stringa i-esima.
     */
    array_di_pointers_a_carattere[i] =
        (char*)malloc(sizeof(char) * 1024);

    /*
     * Leggo una riga da standard input dentro il buffer.
     */
    if (fgets(buffer, 1024, stdin))
    {
        /*
         * Se fgets ha letto anche il carattere di invio,
         * lo sostituisco con il terminatore di stringa.
         */
        buffer[strcspn(buffer, "\n")] = '\0';
    }

    /*
     * Copio il contenuto del buffer nella stringa allocata.
     */
    memcpy(array_di_pointers_a_carattere[i], buffer, 1024);
}

/*
 * Stampo tutte le stringhe memorizzate.
 */
writelines(array_di_pointers_a_carattere, num_arrays);

return 0;
}

void writelines(char *lineptr[], int nlines)
{
    /*
     * lineptr e' un array di puntatori a char.
     * Ogni elemento punta all'inizio di una stringa.
     *
     * Il ciclo continua finche' nlines e' maggiore di zero.
    */

```

```

    */
while (nlines-- > 0)
{
    /*
     * *lineptr indica la stringa corrente.
     * Dopo la stampa, lineptr++ sposta il puntatore
     * alla stringa successiva.
     */
    printf("Valore stringa = %s \n", *lineptr++);
}
}

```

Il programma mostra come gestire dinamicamente un insieme di stringhe in C attraverso un array di puntatori a carattere. La variabile `array_di_pointers_a_carattere` è un doppio puntatore: essa viene utilizzata per rappresentare un array i cui elementi sono puntatori a stringhe. Nel `main` viene innanzitutto allocato dinamicamente lo spazio per contenere cinque puntatori a `char`. Successivamente, per ciascuna posizione dell'array, viene allocato uno spazio di memoria di 1024 caratteri, destinato a contenere una singola stringa inserita dall'utente. La lettura delle stringhe avviene tramite `fgets`, che consente di acquisire una riga da standard input in modo più sicuro rispetto a funzioni come `gets`. Poiché `fgets` può includere anche il carattere di nuova riga `\n`, questo viene rimosso sostituendolo con il terminatore di stringa `\0`. Il contenuto del buffer temporaneo viene poi copiato nella memoria allocata per la stringa corrente tramite `memcpy`. Al termine della lettura, la funzione `writelines` riceve l'array di puntatori e stampa tutte le stringhe memorizzate. All'interno di `writelines`, l'espressione `*lineptr` rappresenta la stringa corrente, mentre l'operazione `lineptr++` sposta il puntatore alla stringa successiva. L'esempio evidenzia quindi come un array di stringhe possa essere gestito attraverso un doppio puntatore e come l'aritmetica dei puntatori permetta di scorrere sequenzialmente le stringhe memorizzate.

Nel parametro di una funzione, le scritture `char **lineptr` e `char *lineptr[]` sono equivalenti. In entrambi i casi, infatti, la funzione riceve un puntatore a puntatore a `char`, cioè un riferimento a una sequenza di puntatori a carattere. Questa equivalenza deriva dal fatto che, quando un array viene passato a una funzione, esso non viene copiato interamente: ciò che viene passato è l'indirizzo del suo primo elemento. Di conseguenza, un parametro scritto come `char *lineptr[]` viene interpretato dal compilatore come `char **lineptr`. Nel caso specifico, ogni elemento di `lineptr` è un puntatore a `char`, cioè l'indirizzo del primo carattere di una stringa. Pertanto, `lineptr` può essere visto come un array di stringhe, oppure, in modo più tecnico, come un puntatore al primo elemento di un array di puntatori a carattere. La notazione

`char *lineptr[]` è spesso più leggibile dal punto di vista didattico, perché suggerisce immediatamente l'idea di una collezione di stringhe. La notazione `char **lineptr`, invece, evidenzia maggiormente la struttura effettiva a livello di memoria, cioè un doppio livello di indirizzione. In entrambi i casi, all'interno della funzione è possibile accedere agli elementi nello stesso modo. Ad esempio, `lineptr[i]` indica la stringa in posizione `i`, mentre `*lineptr` indica la stringa corrente se si sta scorrendo l'array tramite aritmetica dei puntatori. L'istruzione `lineptr++`, invece, sposta il puntatore alla stringa successiva. L'espressione `*lineptr++` deve essere letta tenendo conto della precedenza degli operatori in C. L'operatore di incremento `++` ha precedenza maggiore rispetto all'operatore di dereferenziazione `*`; pertanto l'espressione viene interpretata come `*(lineptr++)`. Questo significa che viene prima utilizzato il valore corrente di `lineptr`, e solo successivamente il puntatore viene incrementato. In altre parole, l'espressione accede alla stringa puntata attualmente da `lineptr` e poi sposta `lineptr` alla stringa successiva. Nel caso dell'istruzione `printf("%s\n", *lineptr++)`, viene quindi stampata la stringa corrente e, subito dopo, il puntatore viene avanzato all'elemento successivo dell'array di stringhe. Questa istruzione è equivalente, dal punto di vista logico, alle due istruzioni seguenti:

```
printf("%s\n", *lineptr);
lineptr++;
```

Questa forma compatta è molto comune in C, ma deve essere letta con attenzione, perché combina in una sola espressione sia l'accesso al valore puntato sia l'avanzamento del puntatore.

2.7 Suddivisione di una stringa con delimitatori multipli

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <math.h>

/*
 * Esempio di input:
 * 192.168.1.2/24
 *
 * Obiettivo:
 * separare la stringa nei valori:
 * 192
 * 168
```



```

* 1
* 2
* 24
*/

int main(int argc, char **argv)
{
    /*
     * Buffer in cui viene salvata la stringa letta da input.
     */
    char buffer[1024];

    /*
     * Puntatore che conterra' di volta in volta
     * il token estratto dalla stringa.
     */
    char *token;

    /*
     * Leggo una riga da standard input.
     */
    if (fgets(buffer, 1024, stdin))
    {
        /*
         * Rimuovo il carattere di newline eventualmente letto da
         fgets.
         */
        buffer[strcspn(buffer, "\n")] = '\0';
    }

    /*
     * Prima chiamata a strtok.
     * La stringa viene divisa usando sia il punto '.'
     * sia lo slash '/' come delimitatori.
     */
    token = strtok(buffer, "./");

    /*
     * Continuo finche' strtok restituisce un token valido.
     */
    while (token != NULL)
    {
        /*

```

```

        * Stampa il token corrente.
        */
printf("Token = %s \n", token);

/*
 * Chiamate successive a strtok.
 * Il primo parametro e' NULL per indicare che si vuole
 * continuare a dividere la stessa stringa precedente.
 */
token = strtok(NULL, "./");
}

return 0;
}

```

Il codice, anche se banale, non è per nulla scontato e serve sempre un esempio del genere, perchè mostra una suddivisione di una stringa tramite la funzione `strtok`. L'obiettivo è prendere una stringa strutturata, come un indirizzo IP con notazione CIDR, ad esempio `192.168.1.2/24`, e separarla nei suoi singoli componenti. La stringa viene inizialmente letta da standard input tramite `fgets` e salvata all'interno del buffer `buffer`. Poiché `fgets` può includere anche il carattere di nuova riga, questo viene rimosso sostituendolo con il terminatore di stringa `\0`. La funzione `strtok` viene poi utilizzata per dividere la stringa in token. In questo caso vengono specificati due delimitatori: il punto `.` e lo slash `/`. Questo significa che ogni volta che `strtok` incontra uno di questi caratteri, interrompe temporaneamente la stringa e restituisce la parte compresa tra due delimitatori. La prima chiamata a `strtok` riceve come parametro il buffer originale, mentre le chiamate successive ricevono `NULL`. Questo comportamento indica alla funzione di continuare la scansione della stessa stringa già iniziata in precedenza. A ogni iterazione del ciclo `while`, il token corrente viene stampato e poi viene richiesto il token successivo. In questo modo, una stringa come `192.168.1.2/24` viene separata nei valori `192`, `168`, `1`, `2` e `24`. L'esempio evidenzia quindi come `strtok` permetta di analizzare una stringa composta da più parti, sfruttando uno o più caratteri delimitatori. È importante ricordare che `strtok` modifica direttamente la stringa originale, sostituendo i delimitatori con il carattere nullo `\0`; per questo motivo, se si vuole conservare la stringa iniziale, è necessario lavorare su una sua copia.

Quando si consulta il manuale della funzione `strtok`, ad esempio tramite il comando `man strtok`, viene mostrata una dichiarazione simile alla seguente:

```
#include <string.h>
```

```
char *
strtok(char *restrict str, const char *restrict sep);
```

La prima riga indica che, per utilizzare `strtok`, è necessario includere la libreria `string.h`. La dichiarazione successiva rappresenta il prototipo della funzione. La funzione `strtok` restituisce un valore di tipo `char *`, cioè un puntatore a carattere. In pratica, restituisce l'indirizzo del primo carattere del token individuato. Se non ci sono più token da estrarre, restituisce `NULL`. Il primo parametro, `char *restrict str`, rappresenta la stringa da suddividere. Nella prima chiamata bisogna passare la stringa originale; nelle chiamate successive, invece, si passa `NULL`, in modo che `strtok` continui a lavorare sulla stessa stringa. Il secondo parametro, `const char *restrict sep`, rappresenta la stringa contenente i caratteri separatori. Ad esempio, se si scrive `"/."`, si sta indicando che sia il punto sia lo slash devono essere considerati delimitatori. La parola chiave `const` indica che la funzione non deve modificare la stringa dei separatori. La parola chiave `restrict`, invece, è un'indicazione per il compilatore: segnala che, durante l'uso della funzione, quei puntatori non dovrebbero sovrapporsi ad altri puntatori usati per accedere alla stessa area di memoria.

In sintesi, il prototipo del manuale ci dice che `strtok` riceve una stringa modificabile e una lista di delimitatori, e restituisce un puntatore al token trovato.

2.8 Esempio compatto/completo con `void *`, `restrict` e puntatori a funzione

```
#include <stdio.h>
#include <stdlib.h>

/*
 * Funzione generica che applica una certa operazione
 * a tutti gli elementi di un array sorgente, salvando
 * i risultati in un array destinazione.
 */
void apply(
    void *restrict dst,           // area di memoria di destinazione
    const void *restrict src,     // area di memoria sorgente, non
    modificabile                  //
    int n,                       // numero di elementi da elaborare
    size_t size,                 // dimensione in byte di ogni
    elemento                      //
)
```

```

    void (*op)(void *restrict, const void *restrict) // funzione da
    applicare
);

/*
 * Funzione che calcola il quadrato di un intero.
 */
void square_int(void *restrict out, const void *restrict in);

int main(void)
{
    /*
     * Array sorgente.
     */
    int input[] = {1, 2, 3, 4, 5};

    /*
     * Numero di elementi dell'array.
     */
    int n = 5;

    /*
     * Allocazione dinamica dell'array di output.
     */
    int *output = malloc(sizeof(int) * n);

    /*
     * Applico la funzione square_int a ogni elemento di input.
     * Il risultato viene scritto dentro output.
     */
    apply(output, input, n, sizeof(int), square_int);

    /*
     * Stampa dell'array risultante.
     */
    for (int i = 0; i < n; i++)
    {
        printf("%d ", output[i]);
    }

    printf("\n");

    /*

```

```

    * Libero la memoria allocata dinamicamente.
    */
    free(output);

    return 0;
}

void apply(
    void *restrict dst,
    const void *restrict src,
    int n,
    size_t size,
    void (*op)(void *restrict, const void *restrict)
)
{
    /*
    * Converto il puntatore generico di destinazione in char*.
    * Questo permette di muoversi in memoria byte per byte.
    */
    char *d = dst;

    /*
    * Converto il puntatore generico sorgente in const char*.
    * Anche qui lo scorrimento avviene byte per byte.
    */
    const char *s = src;

    /*
    * Per ogni elemento, calcolo l'indirizzo corretto
    * nella sorgente e nella destinazione.
    */
    for (int i = 0; i < n; i++)
    {
        /*
        * d + i * size punta alla posizione i-esima dell'output.
        * s + i * size punta alla posizione i-esima dell'input.
        *
        * La funzione op viene poi applicata a questi due indirizzi
        */
        op(d + i * size, s + i * size);
    }
}

```

```

void square_int(void *restrict out, const void *restrict in)
{
    /*
     * Riconverto il puntatore generico sorgente
     * in puntatore a intero costante.
     */
    const int *x = in;

    /*
     * Riconverto il puntatore generico di destinazione
     * in puntatore a intero.
     */
    int *y = out;

    /*
     * Leggo il valore puntato da x, calcolo il quadrato
     * e lo salvo nella posizione puntata da y.
     */
    *y = (*x) * (*x);
}

```

Il codice mostra un esempio avanzato ma compatto di uso dei puntatori in C. L'obiettivo del programma è applicare una certa operazione a ogni elemento di un array sorgente e salvare il risultato in un array di destinazione. Nel caso specifico, l'operazione scelta è il quadrato di un numero intero. Nel `main` viene dichiarato l'array `input`, contenente i valori 1, 2, 3, 4, 5. Successivamente viene dichiarata la variabile `n`, che rappresenta il numero di elementi dell'array. Viene poi allocato dinamicamente un secondo array, `output`, tramite la funzione `malloc`. Questo array conterrà i risultati dell'elaborazione. L'istruzione principale è:

```
apply(output, input, n, sizeof(int), square_int);
```

Essa chiama la funzione `apply`, passando cinque informazioni: l'array di destinazione, l'array sorgente, il numero di elementi, la dimensione di ogni elemento e la funzione da applicare. In questo caso, la funzione da applicare è `square_int`.

La funzione `apply` è progettata in modo generico. I parametri `dst` e `src` sono dichiarati come puntatori generici, rispettivamente `void *` e `const void *`. Un puntatore `void *` può contenere l'indirizzo di un dato di qualunque tipo, ma non può essere dereferenziato direttamente, perché il compilatore non conosce la dimensione del dato puntato. Per questo motivo, all'interno della funzione, i puntatori vengono convertiti in puntatori a `char`.

La conversione in `char *` è fondamentale: in C, l'aritmetica su un puntatore dipende dalla dimensione del tipo puntato. Un puntatore a `int`, quando viene incrementato, avanza di `sizeof(int)` byte; un puntatore a `char`, invece, avanza di un solo byte. Convertendo `dst` e `src` in `char *`, la funzione può calcolare manualmente gli spostamenti in memoria tramite l'espressione `i * size`. Per esempio, nell'istruzione:

```
op(d + i * size, s + i * size);
```

`d + i * size` rappresenta l'indirizzo dell'elemento `i`-esimo dell'array di destinazione, mentre `s + i * size` rappresenta l'indirizzo dell'elemento `i`-esimo dell'array sorgente. Poiché `size` vale `sizeof(int)`, la funzione si sposta correttamente da un intero al successivo. Il parametro `op` è un puntatore a funzione. La sua dichiarazione:

```
void (*op)(void *restrict, const void *restrict)
```

indica che `op` può puntare a una funzione che restituisce `void` e riceve due parametri: un puntatore generico di destinazione e un puntatore generico sorgente costante. Nel programma, `op` viene associato concretamente alla funzione `square_int`. Di conseguenza, ogni chiamata a `op(...)` equivale, in questo esempio, a una chiamata a `square_int(...)`. La funzione `square_int` riceve quindi due indirizzi generici: uno relativo alla posizione dell'array di output in cui scrivere il risultato e uno relativo alla posizione dell'array di input da cui leggere il valore. Al suo interno, il puntatore `in` viene convertito in `const int *`, perché il dato sorgente deve essere letto come intero ma non modificato. Il puntatore `out`, invece, viene convertito in `int *`, perché in quella posizione deve essere scritto il risultato. L'istruzione:

```
*y = (*x) * (*x);
```

legge il valore puntato da `x`, ne calcola il quadrato e salva il risultato nella posizione puntata da `y`. Se `x` punta al valore 3, allora `*x` vale 3 e `*y` riceverà il valore 9. La parola chiave `const`, presente in `const void *src` e in `const void *in`, indica che i dati puntati non devono essere modificati attraverso quel puntatore. Questo rende più chiaro il ruolo del parametro: `src` e `in` rappresentano dati in sola lettura. Il compilatore impedisce quindi modifiche accidentali attraverso quei puntatori. La parola chiave `restrict`, invece, è più sottile. Essa comunica al compilatore che, per tutta la durata dell'utilizzo di quel puntatore, l'area di memoria puntata sarà accessibile principalmente tramite quel puntatore e non tramite altri puntatori concorrenti. In pratica, quando si scrive `void *restrict dst` e `const void *restrict src`, si dichiara l'intenzione che `dst` e `src` non si sovrappongano in memoria. Questo può aiutare il compilatore a ottimizzare il codice, perché può assumere che scrivere in `dst` non modifichi indirettamente ciò che viene letto da `src`. È

importante sottolineare che **restrict** non effettua controlli a runtime: è una promessa fatta dal programmatore al compilatore. Se questa promessa viene violata, ad esempio passando due puntatori che puntano alla stessa area di memoria quando il codice assume che siano separati, il comportamento del programma può diventare non definito. Infine, dopo l'applicazione della funzione, il **main** stampa il contenuto dell'array **output**. Il risultato sarà:

```
1 4 9 16 25
```

Al termine, la memoria allocata dinamicamente viene liberata tramite **free(output)**. Questo passaggio è fondamentale ogni volta che si usa **malloc**, perché evita perdite di memoria. Questo esempio riunisce diversi concetti importanti: puntatori generici, conversione di tipo, aritmetica dei puntatori, puntatori a funzione, uso di **const**, uso di **restrict** e gestione dinamica della memoria. La funzione **apply** è particolarmente interessante perché non dipende direttamente dal tipo di dato elaborato: riceve indirizzi generici, una dimensione in byte e una funzione da applicare. Questo la rende un modello di codice generico e riutilizzabile.

2.9 Albero di puntatori con quattro livelli di indirezione

```
#include <stdio.h>

int main(void)
{
    /*
     * Quattro variabili intere semplici.
     * Queste rappresentano i valori finali,
     * cioè le "foglie" dell'albero.
     */
    int a = 10, b = 20, c = 30, d = 40;

    /*
     * Array di puntatori a intero.
     * Ogni elemento di foglie contiene l'indirizzo
     * di una variabile intera.
     */
    *foglie[0] -> &a
    *foglie[1] -> &b
    *foglie[2] -> &c
    *foglie[3] -> &d
    /*
    int *foglie[4] = {&a, &b, &c, &d};
```



```

/*
 * ramo_sinistro punta al primo elemento dell'array foglie.
 * Quindi permette di raggiungere a e b.
 *
 * ramo_destro punta al terzo elemento dell'array foglie.
 * Quindi permette di raggiungere c e d.
 */
int **ramo_sinistro = &foglie[0];
int **ramo_destro = &foglie[2];

/*
 * albero e' un array di due puntatori a int**.
 *
 * albero[0] contiene l'indirizzo di ramo_sinistro.
 * albero[1] contiene l'indirizzo di ramo_destro.
 *
 * Ogni elemento di albero permette quindi di accedere
 * a uno dei due rami.
 */
int ***albero[2] = {&ramo_sinistro, &ramo_destro};

/*
 * radice e' un puntatore quadruplo.
 * Punta al primo elemento dell'array albero.
 *
 * La catena logica e':
 * radice -> albero -> ramo -> foglia -> valore intero
 */
int ****radice = albero;

/*
 * Accesso al primo valore del ramo sinistro.
 *
 * radice[0] -> primo ramo dell'albero
 * *radice[0] -> ramo_sinistro
 * **radice[0] -> foglie[0], cioe' &a
 * ***radice[0] -> valore di a
 */
printf("a = %d\n", ***radice[0]);

/*
 * Accesso al secondo valore del ramo sinistro.

```

```

    *
    * *radice[0] e' ramo_sinistro, cioe' &foglie[0].
    * (*radice[0] + 1) si sposta a foglie[1].
    * ((*radice[0] + 1) restituisce foglie[1], cioe' &b.
    * ((*(*radice[0] + 1)) restituisce il valore di b.
    */
    printf("b = %d\n", ((*(*radice[0] + 1)));

    /*
    * Accesso al primo valore del ramo destro.
    *
    * radice[1] punta al ramo destro.
    * Seguendo la catena di dereferenziazioni
    * si arriva al valore di c.
    */
    printf("c = %d\n", ***radice[1]);

    /*
    * Accesso al secondo valore del ramo destro.
    *
    * *radice[1] e' ramo_destro, cioe' &foglie[2].
    * (*radice[1] + 1) si sposta a foglie[3].
    * ((*radice[1] + 1) restituisce foglie[3], cioe' &d.
    * ((*(*radice[1] + 1)) restituisce il valore di d.
    */
    printf("d = %d\n", ((*(*radice[1] + 1)));

    return 0;
}

```

Il codice costruisce un esempio di struttura ad albero basata su più livelli di puntatori. L'obiettivo è mostrare in modo esplicito come sia possibile collegare tra loro variabili, puntatori, doppi puntatori, tripli puntatori e un puntatore quadruplo. Le variabili a, b, c e d rappresentano i valori finali della struttura. Esse possono essere viste come le foglie dell'albero, cioè i dati concreti che si vogliono raggiungere. L'array `foglie` contiene quattro puntatori a intero. Ogni elemento dell'array non memorizza direttamente un valore, ma l'indirizzo di una delle variabili intere. Per esempio, `foglie[0]` contiene l'indirizzo di a, mentre `foglie[1]` contiene l'indirizzo di b. Successivamente vengono definiti due rami: `ramo_sinistro` e `ramo_destro`. Il primo punta all'inizio dell'array `foglie`, quindi permette di raggiungere i valori a e b; il secondo punta invece alla posizione `foglie[2]`, quindi permette di raggiungere i valori c e d. L'array `albero` contiene gli indirizzi dei

due rami. In questo modo, `albero[0]` consente di accedere al ramo sinistro, mentre `albero[1]` consente di accedere al ramo destro. A questo livello, non si sta più puntando direttamente ai valori, ma a strutture che a loro volta permettono di raggiungerli. Infine, la variabile `radice` è un puntatore quadruplo. Essa punta al primo elemento dell'array `albero` e rappresenta il punto di ingresso dell'intera struttura. La catena logica può essere descritta come:

```
radice -> albero -> ramo -> foglia -> valore intero
```

Per accedere ai valori, il programma utilizza più dereferenziazioni. Ad esempio, l'espressione `***radice[0]` permette di raggiungere il primo valore del ramo sinistro, cioè `a`. Invece, per raggiungere il secondo valore dello stesso ramo, viene usata l'espressione `*(**radice[0] + 1)`, che prima si sposta alla foglia successiva e poi dereferenzia fino al valore finale. Lo stesso ragionamento viene applicato al ramo destro: `***radice[1]` permette di accedere al valore `c`, mentre `*(**radice[1] + 1)` permette di accedere al valore `d`. Questo esempio mostra come l'aritmetica dei puntatori e la dereferenziazione possano essere combinate per attraversare strutture complesse. Tuttavia, è importante sottolineare che un uso così profondo dei puntatori è raramente consigliabile nel codice reale, perché riduce molto la leggibilità, io l'ho scritto per far capire. Il suo valore principale è didattico: permette di visualizzare chiaramente cosa significa aumentare i livelli di indirezione.

2.10 IMPORTANTE: Ricorsione con puntatori a funzione e puntatori generici `void*`

```
#include <stdio.h>

/*
 * Definisco un nuovo tipo chiamato Operation.
 *
 * Operation rappresenta un puntatore a funzione.
 * La funzione puntata deve:
 * - ricevere un intero;
 * - restituire un intero.
 *
 * Quindi Operation puo' puntare a funzioni come:
 * int funzione(int x)
 */
typedef int (*Operation)(int);
```

```

/*
 * Funzione che riceve un intero e restituisce il suo quadrato.
 * Questa funzione verra' passata come parametro alla funzione
   ricorsiva.
 */
int square(int x)
{
    return x * x;
}

/*
 * Funzione ricorsiva che applica una certa operazione
 * a tutti gli elementi di un array.
 *
 * Parametri:
 * - array: puntatore generico all'array da modificare;
 * - index: posizione corrente dell'array;
 * - size: numero totale di elementi;
 * - op: puntatore alla funzione da applicare.
 */
void recursive_apply(
    void *array,
    int index,
    int size,
    Operation op
)
{
    /*
     * Caso base della ricorsione.
     * Quando index raggiunge size, significa che tutti
     * gli elementi dell'array sono stati elaborati.
     */
    if (index == size)
        return;

    /*
     * Converto il puntatore generico void* in int*.
     * Questo e' necessario per poter trattare la memoria
     * come un array di interi.
     */
    int *p = array;

    /*

```

```

    * *(p + index) accede all'elemento in posizione index.
    *
    * op(*(p + index)) applica la funzione puntata da op
    * al valore corrente.
    *
    * Il risultato viene poi riscritto nella stessa posizione.
    */
    *(p + index) = op(*(p + index));

    /*
    * Chiamata ricorsiva.
    * Si passa allo stesso array, ma alla posizione successiva.
    */
    recursive_apply(array, index + 1, size, op);
}

int main(void)
{
    /*
    * Array di interi su cui lavorare.
    */
    int values[] = {1, 2, 3, 4, 5};

    /*
    * Applico ricorsivamente la funzione square
    * a ogni elemento dell'array values.
    */
    recursive_apply(values, 0, 5, square);

    /*
    * Stampo il contenuto dell'array dopo la modifica.
    */
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", values[i]);
    }

    printf("\n");

    return 0;
}

```

Il codice mostra un esempio di utilizzo combinato di ricorsione, puntatori generici, aritmetica dei puntatori e puntatori a funzione. La prima parte

importante è la dichiarazione:

```
typedef int (*Operation)(int);
```

Questa istruzione definisce un nuovo tipo chiamato `Operation`. Tale tipo rappresenta un puntatore a funzione: più precisamente, un puntatore a una funzione che riceve un parametro di tipo `int` e restituisce un valore di tipo `int`. In questo modo, invece di scrivere ogni volta la forma completa del puntatore a funzione, è possibile usare semplicemente il nome `Operation`. La funzione `square` rispetta esattamente questa forma: riceve un intero e restituisce un intero. Per questo motivo può essere passata come argomento alla funzione `recursive_apply`. La funzione `recursive_apply` riceve quattro parametri: un puntatore generico all'array da modificare, l'indice corrente, la dimensione totale dell'array e il puntatore alla funzione da applicare. Il parametro `array` è di tipo `void *`, quindi può rappresentare genericamente un indirizzo di memoria. Tuttavia, poiché in questo esempio l'array contiene interi, all'interno della funzione viene convertito in `int *` tramite l'istruzione:

```
int *p = array;
```

A questo punto, `p` può essere usato come puntatore al primo elemento dell'array. L'espressione `p + index` calcola l'indirizzo dell'elemento in posizione `index`, mentre `*(p + index)` accede al valore contenuto in quella posizione. L'istruzione centrale è:

```
*(p + index) = op(*(p + index));
```

Essa legge il valore corrente dell'array, lo passa alla funzione puntata da `op` e salva il risultato nella stessa posizione. Nel caso specifico, `op` punta alla funzione `square`; quindi ogni elemento dell'array viene sostituito con il proprio quadrato. La ricorsione è controllata dalla condizione:

```
if (index == size)
    return;
```

Questa rappresenta il caso base. Quando `index` raggiunge `size`, significa che tutti gli elementi sono stati elaborati e la funzione termina. Se invece il caso base non è stato raggiunto, viene eseguita la chiamata ricorsiva:

```
recursive_apply(array, index + 1, size, op);
```

In questo modo la funzione richiama se stessa passando lo stesso array, la stessa operazione, ma l'indice successivo. Il processo continua finché tutti gli elementi sono stati modificati. Nel `main`, l'array `values` viene inizializzato con i valori 1, 2, 3, 4, 5. La chiamata:

```
recursive_apply(values, 0, 5, square);
```

applica la funzione `square` a partire dall'indice 0 fino all'ultimo elemento dell'array. Al termine dell'esecuzione, il contenuto dell'array diventa:

1 4 9 16 25

Questo esempio è utile perché mostra come una funzione possa ricevere un'altra funzione come parametro, rendendo il codice più flessibile. Cambiando la funzione passata a `recursive_apply`, sarebbe possibile applicare operazioni diverse agli stessi dati, senza modificare la logica ricorsiva di attraversamento dell'array.

2.11 Esercizio MIT sui puntatori

Il Massachusetts Institute of Technology mette a disposizione, tramite MIT OpenCourseWare, diversi materiali didattici sul linguaggio C e sui puntatori. Un esempio particolarmente interessante è il file `pointers.c`, appartenente al corso *6.828 Operating System Engineering*. Questo file è pensato come esercizio specifico sui puntatori in C.

Link al materiale:

<https://ocw.mit.edu/courses/6-828-operating-system-engineering-fall-2012/resources/>

2.12 Esempio MIT sull'aritmetica dei puntatori

```
#include <stdio.h>
#include <stdlib.h>

void f(void)
{
    /*
     * Array locale di 4 interi.
     * La memoria viene allocata nello stack.
     */
    int a[4];

    /*
     * Puntatore a intero.
     * malloc(16) alloca 16 byte nello heap.
     * Se un int occupa 4 byte, lo spazio e' sufficiente
     * per 4 interi.
     */
    int *b = malloc(16);
```

```

/*
 * Puntatore a intero non ancora inizializzato.
 * In questo momento contiene un valore indeterminato.
 */
int *c;

int i;

/*
 * Stampa gli indirizzi contenuti in a, b e c.
 * a rappresenta l'indirizzo del primo elemento dell'array.
 * b contiene l'indirizzo restituito da malloc.
 * c, non essendo inizializzato, contiene un valore casuale.
 */
printf("1: a = %p, b = %p, c = %p\n", a, b, c);

/*
 * c viene fatto puntare all'inizio dell'array a.
 * Da questo momento c e a permettono di accedere
 * alla stessa area di memoria.
 */
c = a;

/*
 * Riempie l'array a con i valori:
 * a[0] = 100, a[1] = 101, a[2] = 102, a[3] = 103.
 */
for (i = 0; i < 4; i++)
    a[i] = 100 + i;

/*
 * Poiche' c punta ad a[0], c[0] coincide con a[0].
 */
c[0] = 200;

printf("2: a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
       a[0], a[1], a[2], a[3]);

/*
 * Queste tre istruzioni mostrano tre forme equivalenti
 * di accesso tramite puntatore.
 */

```



```

    * c[1] equivale a *(c + 1).
    * *(c + 2) accede al terzo elemento.
    * 3[c] equivale a c[3], perche' in C:
    * c[3] == *(c + 3)
    * 3[c] == *(3 + c)
    */
c[1] = 300;
*(c + 2) = 301;
3[c] = 302;

printf("3: a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
      a[0], a[1], a[2], a[3]);

/*
 * Il puntatore c viene avanzato di una posizione.
 * Essendo c un int*, c + 1 avanza di sizeof(int) byte.
 * Ora c punta ad a[1].
 */
c = c + 1;

/*
 * Modifica il valore puntato da c.
 * Poiche' c punta ad a[1], viene modificato a[1].
 */
*c = 400;

printf("4: a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
      a[0], a[1], a[2], a[3]);

/*
 * Qui c viene prima convertito in char*.
 * Un char* avanza di un solo byte alla volta.
 *
 * ((char *) c + 1) sposta l'indirizzo di un solo byte,
 * non di sizeof(int) byte.
 *
 * Poi il risultato viene riconvertito in int*.
 * Questo produce un puntatore non allineato rispetto
 * agli interi dell'array.
 */
c = (int *) ((char *) c + 1);

/*

```

```

    * Scrivere attraverso questo puntatore e' pericoloso:
    * si sta scrivendo un int a partire da un indirizzo
    * spostato di un solo byte.
    *
    * Questo puo' sovrascrivere parzialmente piu' elementi
    * dell'array e il comportamento non e' portabile.
    */
*c = 500;

printf("5: a[0] = %d, a[1] = %d, a[2] = %d, a[3] = %d\n",
      a[0], a[1], a[2], a[3]);

/*
 * b viene fatto puntare ad a + 1.
 * Poiche' a viene convertito in int*,
 * b puntera' ad a[1].
 */
b = (int *) a + 1;

/*
 * c viene fatto puntare a un indirizzo spostato
 * di un solo byte rispetto all'inizio di a.
 */
c = (int *) ((char *) a + 1);

/*
 * Stampa gli indirizzi finali.
 * b sara' spostato di sizeof(int) byte rispetto ad a.
 * c sara' spostato di un solo byte rispetto ad a.
 */
printf("6: a = %p, b = %p, c = %p\n", a, b, c);
}

int main(int ac, char **av)
{
    /*
     * Chiamata alla funzione dimostrativa.
     */
    f();

    return 0;
}

```

Output:

```

1: a = 0x7ff7b232be90, b = 0x6000036f4040, c = 0x7ff8189842f2
2: a[0] = 200, a[1] = 101, a[2] = 102, a[3] = 103
3: a[0] = 200, a[1] = 300, a[2] = 301, a[3] = 302
4: a[0] = 200, a[1] = 400, a[2] = 301, a[3] = 302
5: a[0] = 200, a[1] = 128144, a[2] = 256, a[3] = 302
6: a = 0x7ff7b232be90, b = 0x7ff7b232be94, c = 0x7ff7b232be91

```

L'output mostra passo dopo passo gli effetti dell'aritmetica dei puntatori sull'array `a`. Nella prima riga vengono stampati gli indirizzi di `a`, `b` e `c`. L'indirizzo di `a` corrisponde all'inizio dell'array locale, mentre `b` contiene l'indirizzo restituito da `malloc`. Il valore di `c`, invece, non è significativo, perché il puntatore non è ancora stato inizializzato. Dopo l'assegnazione `c = a`, il puntatore `c` punta al primo elemento dell'array `a`. Per questo motivo, quando viene eseguita l'istruzione `c[0] = 200`, viene modificato direttamente `a[0]`. L'output della seconda riga conferma infatti che il primo elemento dell'array è diventato 200, mentre gli altri elementi conservano i valori iniziali 101, 102 e 103. La terza riga dell'output mostra l'effetto di tre forme equivalenti di accesso agli elementi tramite puntatore. L'istruzione `c[1] = 300` modifica `a[1]`, l'istruzione `*(c + 2) = 301` modifica `a[2]`, mentre `3[c] = 302` modifica `a[3]`. Quest'ultima forma è insolita, ma valida in C, perché `c[3]` equivale a `*(c + 3)` e `3[c]` equivale a `*(3 + c)`. Successivamente, l'istruzione `c = c + 1` sposta il puntatore `c` di una posizione avanti. Poiché `c` è un puntatore a `int`, l'avanzamento non è di un singolo byte, ma di `sizeof(int)` byte. Di conseguenza, `c` punta ora ad `a[1]`. Quando viene eseguito `*c = 400`, il valore di `a[1]` viene sostituito con 400, come mostrato nella quarta riga dell'output. La quinta riga è la più importante dal punto di vista didattico. L'istruzione `c = (int *) ((char *) c + 1)` converte temporaneamente `c` in un puntatore a `char`, lo sposta avanti di un solo byte e poi lo riconverte in puntatore a `int`. In questo modo `c` non punta più correttamente all'inizio di un intero, ma a un indirizzo interno alla rappresentazione binaria dell'array. Scrivere `*c = 500` in quella posizione produce quindi un risultato anomalo: `a[1]` diventa 128144 e `a[2]` diventa 256. Questo accade perché la scrittura di un intero a partire da un indirizzo non allineato modifica i byte appartenenti a più elementi dell'array. Infine, l'ultima riga confronta gli indirizzi finali. L'indirizzo di `a` è `0x7ff7b232be90`. Il puntatore `b`, ottenuto con `(int *) a + 1`, vale `0x7ff7b232be94`: esso è avanzato di quattro byte, cioè della dimensione di un `int` su questa macchina. Il puntatore `c`, ottenuto con `(char *) a + 1`, vale invece `0x7ff7b232be91`: esso è avanzato di un solo byte. Questo confronto chiarisce la differenza fondamentale tra aritmetica dei puntatori su `int *` e aritmetica dei puntatori su `char *`.

2.13 Stanford University - Scambio degli estremi di un array tramite aritmetica dei puntatori

```
#include <stdio.h>
#include <stdlib.h>

/*
 * Funzione che scambia il primo e l'ultimo elemento
 * di un array di interi.
 */
void swap_ends_int(int *arr, size_t nelems)
{
    /*
     * Salvo temporaneamente il primo elemento dell'array.
     * *arr equivale ad arr[0].
     */
    int tmp = *arr;

    /*
     * L'ultimo elemento si trova alla posizione nelems - 1.
     * *(arr + nelems - 1) equivale ad arr[nelems - 1].
     */
    *arr = *(arr + nelems - 1);

    /*
     * Completo lo scambio assegnando il valore iniziale
     * del primo elemento all'ultima posizione.
     */
    *(arr + nelems - 1) = tmp;
}

/*
 * Stessa logica, ma applicata a un array di long.
 */
void swap_ends_long(long *arr, size_t nelems)
{
    long tmp = *arr;
    *arr = *(arr + nelems - 1);
    *(arr + nelems - 1) = tmp;
}

int main(int argc, char **argv)
{
```

```

/*
 * Array di interi.
 */
int i_array[] = {10, 40, 80, 20, -30, 50};

/*
 * Calcolo del numero di elementi dell'array.
 * sizeof(i_array) restituisce la dimensione totale in byte.
 * sizeof(i_array[0]) restituisce la dimensione di un elemento.
 */
size_t i_nelems = sizeof(i_array) / sizeof(i_array[0]);

/*
 * Scambio del primo e dell'ultimo elemento dell'array di int.
 */
swap_ends_int(i_array, i_nelems);

/*
 * Array di long.
 */
long l_array[] = {100, 400, 800, 200, -300, 500};

/*
 * Calcolo del numero di elementi dell'array di long.
 */
size_t l_nelems = sizeof(l_array) / sizeof(l_array[0]);

/*
 * Scambio del primo e dell'ultimo elemento dell'array di long.
 */
swap_ends_long(l_array, l_nelems);

return 0;
}

```

Il codice mostra come scambiare il primo e l'ultimo elemento di un array utilizzando l'aritmetica dei puntatori. Sono presenti due funzioni quasi identiche: `swap_ends_int`, che lavora su un array di `int`, e `swap_ends_long`, che lavora su un array di `long`. La funzione `swap_ends_int` riceve un puntatore al primo elemento dell'array e il numero totale di elementi. Il parametro `arr` punta quindi all'inizio dell'array. Per questo motivo, l'espressione `*arr` equivale ad accedere al primo elemento, cioè `arr[0]`. Per raggiungere l'ultimo elemento, viene usata l'espressione `arr + nelems - 1`. Poiché `arr` è un pun-

tatore a `int`, l'operazione `arr + k` non avanza di `k` byte, ma di `k` elementi di tipo `int`. Di conseguenza, `*(arr + nelems - 1)` equivale a `arr[nelems - 1]`. Lo scambio avviene tramite una variabile temporanea `tmp`. Prima viene salvato il valore del primo elemento, poi il primo elemento viene sostituito con l'ultimo, e infine l'ultimo elemento riceve il valore iniziale del primo. Nel `main`, il numero di elementi dell'array viene calcolato tramite:

```
sizeof(i_array) / sizeof(i_array[0])
```

Questa espressione divide la dimensione totale dell'array per la dimensione di un singolo elemento, ottenendo così il numero effettivo di elementi presenti. L'aspetto didatticamente interessante del codice è che la stessa logica viene duplicata per due tipi diversi, `int` e `long`. Questo mostra un limite del C: senza usare puntatori generici `void *`, oppure macro, è necessario scrivere funzioni diverse per tipi diversi, anche quando l'algoritmo è concettualmente identico. Il codice è quindi utile per comprendere sia l'aritmetica dei puntatori, sia il rapporto tra array e puntatori in C. Quando un array viene passato a una funzione, esso decade a puntatore al suo primo elemento; per questo motivo la funzione non conosce automaticamente la lunghezza dell'array e deve riceverla come parametro separato.

2.14 Dartmouth College - Array di strutture e aritmetica dei puntatori

```
#include <stdio.h>
```

```
#define NUMPEOPLE 100
```

```
/*
 * Definizione di una struttura person.
 *
 * La struttura contiene:
 * - un puntatore a char per il nome;
 * - un puntatore a char per l'indirizzo;
 * - un intero per l'eta'.
 *
 * Con typedef possiamo usare direttamente il nome person
 * senza dover scrivere ogni volta struct _person.
 */
typedef struct _person {
    char *name;
    char *addr;
    int age;
```

```

} person;

/*
 * Array globale di 100 strutture person.
 */
person people[NUMPEOPLE];

/*
 * Puntatore a una struttura person.
 */
person *p;

/*
 * Variabile globale usata per accumulare le eta'.
 * Essendo globale, viene inizializzata automaticamente a 0.
 */
int age;

/*
 * Stringhe costanti usate per inizializzare nome e indirizzo.
 */
char *myname = "Andrew T. Campbell";
char *myadr = "People's Republic of Norwich";

int main(void)
{
    /*
     * p viene inizializzato con l'indirizzo del primo elemento
     * dell'array people.
     *
     * In C, il nome di un array usato in questo contesto
     * decade a puntatore al suo primo elemento.
     */
    p = people;

    int i;

    /*
     * sizeof(p) restituisce la dimensione del puntatore p.
     * sizeof(person) restituisce la dimensione di una singola
     * struttura person.
     */
    printf("pointer needs %ld bytes, struct person needs %u bytes\n"

```

```

,
    sizeof(p), (unsigned int)sizeof(person));

/*
 * &p e' l'indirizzo della variabile puntatore p.
 * p, invece, e' il valore contenuto dentro p,
 * cioe' l'indirizzo a cui p sta puntando.
 */
printf("The address of p is %p, it points to %p\n",
       (void *)&p, (void *)p);

/*
 * people rappresenta l'indirizzo del primo elemento dell'array.
 * Quindi, in questo momento, people e p hanno lo stesso valore.
 */
printf("The address of people is %p\n", (void *)people);

/*
 * Incremento del puntatore.
 *
 * Poiche' p e' un puntatore a person, p++ non avanza di un byte
,
 * ma di sizeof(person) byte.
 *
 * Dopo questa istruzione, p punta a people[1].
 */
p++;

printf("after incrementing p its value is %p\n", (void *)p);

/*
 * Decremento del puntatore.
 *
 * p torna a puntare a people[0].
 */
p--;

printf("after decrementing p its value is %p\n", (void *)p);

/*
 * Inizializzazione dell'array di strutture.
 *
 * Ogni elemento dell'array riceve:

```



```

    * - eta' uguale a 21;
    * - lo stesso puntatore alla stringa myname;
    * - lo stesso puntatore alla stringa myadr.
    */
for (i = 0; i < NUMPEOPLE; i++) {
    people[i].age = 21;
    people[i].name = myname;
    people[i].addr = myadr;
}

/*
 * Reinizializzo p all'inizio dell'array.
 */
p = people;

/*
 * Calcolo della somma delle eta'.
 *
 * Si attraversa l'array usando il puntatore p.
 * L'operatore -> permette di accedere a un campo
 * di una struttura tramite puntatore.
 *
 * p->age equivale a (*p).age.
 */
for (i = 0; i < NUMPEOPLE; i++) {
    age += p->age;
    p++;
}

/*
 * Riporto p all'inizio dell'array per stampare
 * i dati della prima persona.
 */
p = people;

printf("%s is %d years old and lives in %s\n",
        p->name, p->age, p->addr);

printf("The accumulative age of all people is %d years\n", age);

return 0;
}

```

Output:

```

pointer needs 4 bytes, struct people 0xc (HEX) bytes
The address of p is 0x2070, it's contents are (i.e., it points to) 0
    x2080
The address of people is 0x2080
after incrementing p its value is 0x208c
after decrementing p its value is 0x2080
Andrew T. Campbell is 21 years old (again) and lives in People's
    Republic of Norwich
The accumulative age of all people is 2100 years

```

Il codice mostra l'utilizzo combinato di strutture, array di strutture e puntatori a struttura. La struttura `person` rappresenta una persona ed è composta da tre campi: un puntatore a carattere per il nome, un puntatore a carattere per l'indirizzo e un intero per l'età. L'istruzione `person people[NUMPEOPLE]`; dichiara un array globale di 100 elementi, ognuno dei quali è una struttura di tipo `person`. La variabile `p`, invece, è un puntatore a `person`: essa può quindi contenere l'indirizzo di una struttura `person`. Nel `main`, l'assegnazione `p = people;` fa puntare `p` al primo elemento dell'array. In C, infatti, quando il nome di un array viene usato in un'espressione, esso decade generalmente a puntatore al primo elemento. Di conseguenza, `people` equivale all'indirizzo di `people[0]`. Una parte importante del codice riguarda la differenza tra `&p` e `p`. L'espressione `&p` indica l'indirizzo della variabile puntatore `p`; l'espressione `p`, invece, indica il contenuto della variabile `p`, cioè l'indirizzo della struttura a cui essa punta. Quando viene eseguito `p++`, il puntatore non avanza di un singolo byte, ma di `sizeof(person)` byte. Questo accade perché `p` è un puntatore a `person`; quindi l'aritmetica dei puntatori tiene conto della dimensione del tipo puntato. Dopo `p++`, il puntatore si sposta da `people[0]` a `people[1]`. Con `p--`, invece, torna alla posizione precedente. Il ciclo `for` inizializza tutti gli elementi dell'array. Ogni persona riceve età pari a 21, lo stesso nome e lo stesso indirizzo. È importante osservare che i campi `name` e `addr` non contengono direttamente le stringhe, ma puntano alle stringhe definite nelle variabili `myname` e `myadr`. Successivamente, il puntatore `p` viene usato per attraversare l'intero array. A ogni iterazione, l'espressione `p->age` accede al campo `age` della struttura puntata da `p`. L'operatore `->` è una forma compatta equivalente a `(*p).age`. Dopo aver letto l'età, il puntatore viene incrementato con `p++`, passando alla struttura successiva. Alla fine, il programma stampa i dati della prima persona e la somma complessiva delle età. Poiché ci sono 100 persone e ciascuna ha età 21, il risultato accumulato sarà 2100. Questo esempio è utile perché chiarisce tre concetti fondamentali: il decadimento degli array a puntatori, l'aritmetica dei puntatori su strutture e l'utilizzo dell'operatore `->` per accedere ai campi di una struttura tramite puntatore.

3 Conclusione

Questo documento nasce con l'obiettivo di rendere più chiaro e concreto uno degli argomenti più importanti del linguaggio C: i puntatori. Attraverso esempi progressivi, frammenti di codice commentati e spiegazioni mirate, si è cercato di mostrare non solo la sintassi, ma soprattutto il ragionamento che si trova dietro l'uso degli indirizzi di memoria. Se il contenuto di questo documento viene compreso con attenzione, i puntatori possono considerarsi solidamente associati nei loro aspetti fondamentali e avanzati. Sono stati infatti affrontati puntatori semplici, doppi puntatori, puntatori generici, puntatori a funzione, aritmetica dei puntatori, array, stringhe, strutture, memoria dinamica e livelli più complessi di indirizzione. Questo lavoro non nasce dal nulla e non è il risultato di una semplice raccolta casuale di esempi. È stato costruito nel tempo, attraverso anni di studio, esercizi, errori, revisioni e approfondimenti. Ogni sezione è stata pensata con finalità didattica, cercando di trasformare concetti spesso percepiti come astratti o difficili in esempi leggibili, commentati e progressivi. Naturalmente, la piena padronanza dei puntatori richiede pratica. Tuttavia, comprendere davvero gli esempi proposti significa acquisire una base molto solida per leggere, scrivere e analizzare codice C con maggiore consapevolezza. L'obiettivo finale non è imparare a memoria le istruzioni, ma sviluppare la capacità di capire cosa accade realmente in memoria quando un programma viene eseguito.